

Este archivo define la **estructura de datos** de una instrucción TAC. Cada instrucción es un objeto con cinco campos internos. No ejecuta nada por sí solo; solo *describe* una operación para que luego **CodigoTresDir** la emita y la imprima.

Los cinco campos internos

```
private final Tipo    tipo;           // ¿qué tipo de instrucción es?
private final String  resultado;      // dónde se guarda el resultado
private final String  arg1;           // primer operando
private final String  operador;       // símbolo de la operación (+, -, >, etc.)
private final String  arg2;           // segundo operando
```

No todos los campos se usan en todos los tipos. Los que no aplican se guardan como **null**. La siguiente tabla muestra qué campo tiene valor (✓) y cuál queda **null** (—) para cada tipo:

Tipo	resultado	arg1	operador	arg2	Ejemplo impreso
ASIGNAR	destino	fuelle	—	—	x = 10
BINARIA	destino	izquierda	operador	derecha	t1 = a + b
UNARIA	destino	operando	operador	—	t1 = -x
ETIQUETA	nombre	—	—	—	L1:
SALTO_INCOND	etiqueta	—	—	—	goto L1
SALTO_COND_FALSO	etiqueta	condición	—	—	if_false t1 goto L1
PARAM	—	argumento	—	—	param x
LLAMADA	destino	función	—	#args	t2 = call suma, 2
RETORNO	—	valor (o null)	—	—	return t1 / return
IMPRIMIR	—	valor	—	—	print z
INICIO_FUNC	nombre	—	—	—	begin_func suma
FIN_FUNC	nombre	—	—	—	end_func suma

Constructor privado y métodos de fábrica

El constructor de **Instruccion** es **private**. Esto significa que **nadie puede crear una instrucción directamente** con **new Instruccion(...)**. La única forma de crear una es usando los métodos de fábrica estáticos:

```
Instruccion.asignar("x", "10")           → x = 10
Instruccion.binaria("t1", "a", "+", "b") → t1 = a + b
Instruccion.etiqueta("L1")               → L1:
Instruccion.saltoCondFalso("t1", "L1")   → if_false t1 goto L1
```

Esto protege la clase: es imposible crear una instrucción **BINARIA** sin operador, o una **ETIQUETA** con un **arg2** que no tiene sentido.

El método `toString()`

Es el encargado de convertir el objeto en texto para imprimirlo. Usa un **switch** sobre el tipo y construye la cadena con los campos que corresponden a cada caso:

```
case BINARIA:
    return resultado + " = " + arg1 + " " + operador + " " + arg2;
    //      "t1"           "="   "a"      " "   "+"      " "   "b"
    //  → imprime: "t1 = a + b"

case SALTO_COND_FALSO:
    return "if_false " + arg1 + " goto " + resultado;
    //              "t1"              "L1"
    //  → imprime: "if_false t1 goto L1"

case RETORNO:
    if (arg1 != null) return "return " + arg1;
    else              return "return";
    //  si arg1 es null significa retorno vacío (función void)
```

CodigoTresDir.java — ¿Qué hace este archivo?

Este archivo es el **generador**. Recorre el árbol de parseo usando el patrón Visitor (igual que **SemanticAnalyzer**) y va creando objetos **Instruccion** a medida que visita cada nodo. Al terminar, tiene una lista con todas las instrucciones del programa en orden.

Herencia y patrón Visitor

```
public class CodigoTresDir extends MiLenguajeBaseVisitor<String>
```

MiLenguajeBaseVisitor<String> es generado automáticamente por ANTLR. El tipo genérico **<String>** define qué devuelve cada método **visit**. En este caso, los métodos devuelven un **String** que representa el **"lugar"** donde quedó el resultado de una expresión: puede ser un temporal ("**t1**"), el nombre de una variable ("**x**"), o un literal ("**10**", "**true**").

Los tres elementos centrales

```
private final List<Instruccion> instrucciones = new ArrayList<>();
```

La lista donde se acumulan todas las instrucciones generadas. **emitir()** agrega al final de esta lista.

```
private int contTemp = 0;
private int contEtiq = 0;
```

Contadores que garantizan nombres únicos. Cada vez que se necesita un nuevo temporal o etiqueta, el contador sube en uno y nunca se repite.

```
private String nuevaTemp()    { return "t" + (++contTemp); } // t1, t2, t3 ...
private String nuevaEtiqueta() { return "L" + (++contEtiq); } // L1, L2, L3 ...
private void emitir(Instruccion i) { instrucciones.add(i); }
```

Cómo funciona `imprimir()`

Recorre la lista de instrucciones y las imprime numeradas. Las etiquetas (`L1:`, `L2:`) no reciben número porque son marcadores de posición, no operaciones:

```
for (Instruccion inst : instrucciones) {
    if (inst.getTipo() == Instruccion.Tipo.ETIQUETA) {
        System.out.println(" " + inst);           // " L1:" sin número
    } else {
        System.out.printf(" (%3d) %s\n", numero++, inst); // " ( 1) x = 10"
    }
}
```

Los métodos `visit` — sentencias vs expresiones

Los métodos se dividen en dos grupos con comportamientos distintos:

Sentencias (devuelven `null`): no tienen valor, solo producen efectos (emitir instrucciones).

```
visitDeclaracion    → emite ASIGNAR si hay inicializador
visitAsignacion     → emite ASIGNAR
visitSentenciaCout   → emite IMPRIMIR
visitSentenciaIf     → emite saltos condicionales y etiquetas
visitSentenciaWhile → emite etiquetas y saltos para el ciclo
visitDeclaracionFuncion → emite INICIO_FUNC, cuerpo, FIN_FUNC
visitSentenciaReturn → emite RETORNO con o sin valor
```

Expresiones (devuelven un `String` — el "lugar" del resultado):

```
visitExprAditiva      → crea tN, emite BINARIA, devuelve "tN"
visitExprRelacional   → crea tN, emite BINARIA, devuelve "tN"
visitExprNot          → crea tN, emite UNARIA, devuelve "tN"
visitExprIdentificador → devuelve el nombre de la variable directamente (sin
```

```

crear tN)
visitExprEntero      → devuelve el número como texto (sin crear tN)
visitExprCadena      → devuelve la cadena como texto (sin crear tN)
visitExprAgrupada    → delega al hijo, devuelve lo que devuelva el hijo

```

La clave está en que **los literales y las variables no crean temporales**. Solo las *operaciones* crean uno. Esto evita instrucciones redundantes como `t1 = 10` seguida de `x = t1` cuando basta con `x = 10`.

Generación de `if` y `while` — cómo se usan las etiquetas

`if sin else:`

```

Evaluar condición → tN
if_false tN goto L_fin      ← si es falsa, saltar todo el bloque
    ...bloque then...
L_fin:

```

`if con else:`

```

Evaluar condición → tN
if_false tN goto L_else    ← si es falsa, ir al else
    ...bloque then...
goto L_fin                 ← saltar el else (ya ejecutamos el then)
L_else:
    ...bloque else...
L_fin:

```

`while:`

```

L_inicio:                  ← punto de retorno al inicio del ciclo
Evaluar condición → tN
if_false tN goto L_fin     ← si es falsa, salir del ciclo
    ...cuerpo...
goto L_inicio              ← volver a evaluar la condición
L_fin:

```

Generación de llamadas a función

```

// 1. Visitar cada argumento y emitir un PARAM por cada uno
for (ExpresionContext arg : args) {
    String lugar = visit(arg);
    emitir(Instruccion.param(lugar));
}
// 2. Emitir el LLAMADA con el número de argumentos
String t = nuevaTemp();

```

```
emitir(Instruccion.llamada(t, nombreFuncion, String.valueOf(args.size())));  
return t; // devolver el temporal con el valor de retorno
```

Los **param** se emiten antes del **call** para que el receptor sepa cuántos valores leer. El campo **numArgs** en **LLAMADA** indica exactamente cuántos **param** le preceden.

Ejemplo paso a paso

Código de entrada (**ejemplo.txt**)

```
int x = 10;  
int y = 3;  
  
int suma(int a, int b) {  
    return a + b;  
}  
  
if(x > y){  
    cout << "es mayor x";  
}  
int z = suma(x, y);  
cout << z;
```

Paso 1 — Declaraciones simples

```
int x = 10;  
int y = 3;
```

visitDeclaracion detecta que hay un inicializador y llama a **visit(expresion)**.

Los literales enteros (**visitExprEntero**) devuelven el texto **"10"** directamente sin crear temporal. Con ese lugar se emite la asignación.

```
( 1) x = 10  
( 2) y = 3
```

Paso 2 — Declaración de función

```
int suma(int a, int b) {  
    return a + b;  
}
```

`visitDeclaracionFuncion` emite `begin_func`, luego llama a `visitChildren` sobre el bloque (no a `visitBloque`, para no crear un scope extra). Al terminar emite `end_func`.

Dentro del cuerpo, `visitSentenciaReturn` visita `a + b`:

- `visitExprAditiva` llama a `visit` sobre `a` y `b` por separado.
- `visitExprIdentificador` devuelve "`a`" y "`b`" directamente.
- `visitExprAditiva` crea `t1`, emite `t1 = a + b`, devuelve "`t1`".
- Con ese lugar, `visitSentenciaReturn` emite `return t1`.

```
( 3) begin_func suma
( 4) t1 = a + b
( 5) return t1
( 6) end_func suma
```

Paso 3 — `if` sin `else`

```
if(x > y){
    cout << "es mayor x";
}
```

`visitSentenciaIf` detecta un solo bloque (sin `else`) y reserva una etiqueta `L1` de salida. Luego:

1. Visita la condición: `visitExprRelacional` → crea `t2`, emite `t2 = x > y`, devuelve "`t2`".
2. Si `t2` es **falso**, salta al fin: `if_false t2 goto L1`.
3. Ejecuta el cuerpo: `visitExprCadena` devuelve "`es mayor x`" directamente → `print "es mayor x"`.
4. Emite la etiqueta de salida: `L1:`.

```
( 7) t2 = x > y
( 8) if_false t2 goto L1
( 9) print "es mayor x"
L1:
```

Paso 4 — Llamada a función

```
int z = suma(x, y);
```

`visitDeclaracion` visita la expresión de inicialización, que es una `exprLlamada`.

`visitExprLlamada` emite un `param` por cada argumento en orden, luego el `call` a un nuevo temporal `t3`, y devuelve "`t3`". De vuelta en `visitDeclaracion` se emite la asignación final.

```
( 10) param x
( 11) param y
( 12) t3 = call suma, 2
( 13) z = t3
```

Paso 5 — `cout` final

```
cout << z;
```

`visitSentenciaCout` visita la expresión. `visitExprIdentificador` devuelve "z" directamente sin crear temporal.

```
( 14) print z
```

Salida completa de la Fase 4

```
=== FASE 4: CÓDIGO DE TRES DIRECCIONES ===

( 1) x = 10
( 2) y = 3
( 3) begin_func suma
( 4) t1 = a + b
( 5) return t1
( 6) end_func suma
( 7) t2 = x > y
( 8) if_false t2 goto L1
( 9) print "es mayor x"
L1:
(10) param x
(11) param y
(12) t3 = call suma, 2
(13) z = t3
(14) print z
```

Reglas de generación de temporales

Construcción	¿Se crea temporal?	Razón
Literal (<code>10</code> , <code>3.14</code> , <code>'a'</code> , <code>"hola"</code> , <code>true</code>)	No	Su valor ya es conocido; se usa directamente
Identificador (<code>x</code> , <code>z</code>)	No	El nombre de la variable ya es el "lugar"
Expresión binaria (<code>a + b</code> , <code>x > y</code>)	Sí (<code>tN</code>)	El resultado necesita un nombre para usarse después
Expresión unaria (<code>-x</code> , <code>!cond</code>)	Sí (<code>tN</code>)	Igual que binaria
Llamada a función (<code>suma(x, y)</code>)	Sí (<code>tN</code>)	El valor de retorno necesita un lugar
Expresión agrupada (<code>(a + b)</code>)	No	Solo delega al hijo; los paréntesis ya los resolvió el parser

Cómo ejecutar

```
cd demo
mvn clean package
java -jar target/demo-1.0-jar-with-dependencies.jar ejemplo.txt
```